

Producing an FM Radio Tuner

Turtushig Dashjav and Meba Habte

January 29, 2026

Abstract

Capitalizing on the TEA5767 FM Stereo Radio module’s ability to pick up frequency signals and its built-in headphone amplifier, we created an Arduino circuit that tunes into FM radio channels and plays the audio through a headphone. We have two modes of input: PRESET, which allows the user to move between six known channel frequencies, and POT, which uses a potentiometer to vary the frequency. The LCD was used to make the circuit friendlier for the user by displaying what frequency one’s on and the mode they’re using.

1 Introduction

Electromagnetic waves are oscillations of magnetic and electric fields propagating through space at the speed of light. These waves, unlike mechanical waves, don’t require a medium to propagate. To characterize an electromagnetic wave, the following relationship between its frequency and wavelength with the speed of light is used: $c = f\lambda$.

Information is transferable through modulation—encoding of information onto a wave through the variation of either its amplitude or frequency [1]. FM (Frequency Modulation) encodes audio information into electromagnetic waves. When a radio receives this changed signal, it extracts information from the wave to determine the message that was sent. This also requires the radio to selectively respond to one frequency while ignoring others.

This project implements the Arduino UNO and a TEA5767 radio module to detect and demodulate these electromagnetic waves, allowing the user to listen to the audio output. (To see the code, consult the Appendix: Code).

2 Components

Table 1: Component List for Arduino FM Radio

Component	Qty	Purpose
Arduino UNO	1	Microcontroller board to control the radio
TEA5767 Module	1	Tunable FM Radio Chip
16x2 LCD Display	1	Displays current input mode and frequency
Potentiometer	1	Analog input for manual frequency tuning

Resistor 1k Ω	1	Prevents LCD from drawing excessive current
Resistor 220 Ω	1	Prevents LCD from drawing excessive current
Pushbutton	2	Mode change and Preset step control
9V Battery	1	Supplies the Arduino and the circuit

The component list is based on [2]. (Note that we used headphones for the audio output instead of a speaker amplifier).

3 Circuit Description

At the heart of our circuit lies the Arduino. The Arduino receives information from the human-input components (the pushbuttons and the potentiometer) to specify a frequency or mode, and commands other components (LCD, TEA5767) to display input values and tune into FM channels.

POT mode: The potentiometer is a device that gives a variable resistance. The outer pins of the potentiometer are connected to the 5V rail and the ground rail. This allows the potentiometer to output a variable voltage (0-5V), which is then read by the Arduino to map it into the FM frequency band. This process is made possible by the Analog-to-Digital Converter inside the Arduino, which determines the voltage change (analog) and translates it into frequency increments (digital). Then, the radio module tunes into that frequency, demodulates the signal, and produces it as audio through the headphones. The last component of the circuit, the LCD,

is simply used as a tool to display the current mode and frequency.

PRESET mode: When the radio is turned on, the device is in the aforementioned POT mode. However, if the user pushes the mode button, the software reads that as LOW, causing the device to operate in the PRESET mode. In the PRESET mode, the second button is used to tune into one of six preset FM channels (88.1, 92.5, 94.5, 96.5, 102.3, 107.5 MHz).

The circuit diagram and a photo of the device can be seen from Figures 1 and 2.

4 Conclusion and Future Improvements

This project demonstrates the use of modular design, state machines, and I²C communication to control hardware reliably. Though our radio device works and the audio quality is decent, it has limitations as well. It currently has no way to adjust the volume of the audio output. Furthermore, it is unable to output audio through speakers, as the TEA5767 module does not have a built-in speaker amplifier. Additional improvements to this project include adding a dual-gang potentiometer as the volume adjustment knob, which would require soldering it to the module itself. Adding a feature to switch between headphones and external speakers is another improvement.

Ultimately, this project allowed us to both exercise and expand our conceptual understanding of electronics to practical circuitry. Throughout its development, we faced many challenges due to the greater involvement we had in its design. This allowed us to develop an affinity for problem-solving and troubleshooting, which is the main objective of this course.

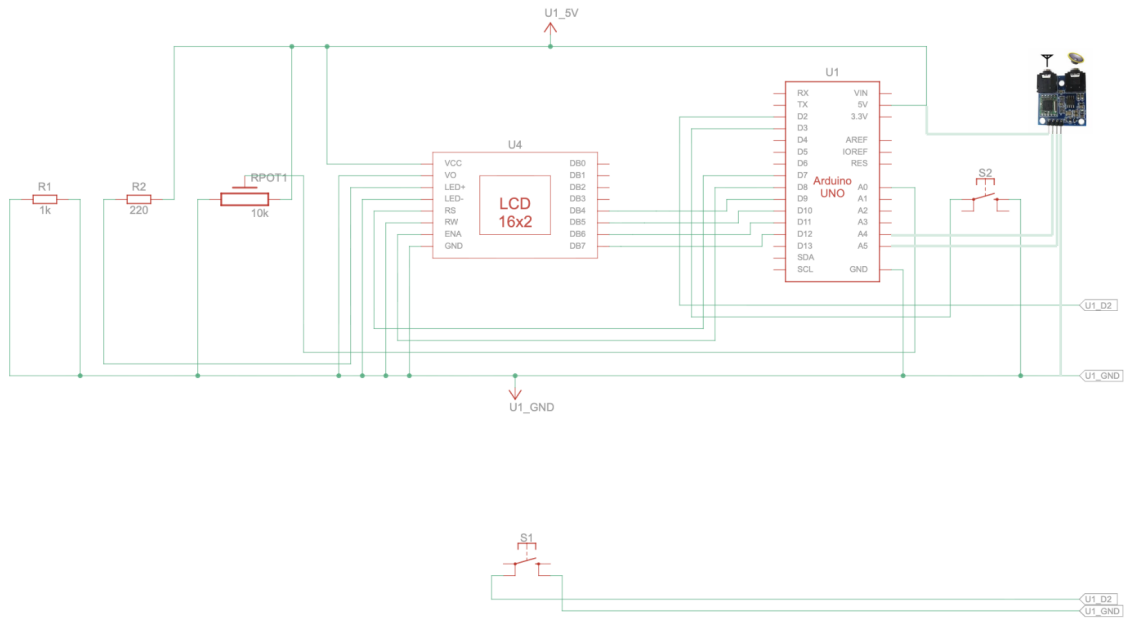


Figure 1: Schematic of the Arduino FM Radio Circuit

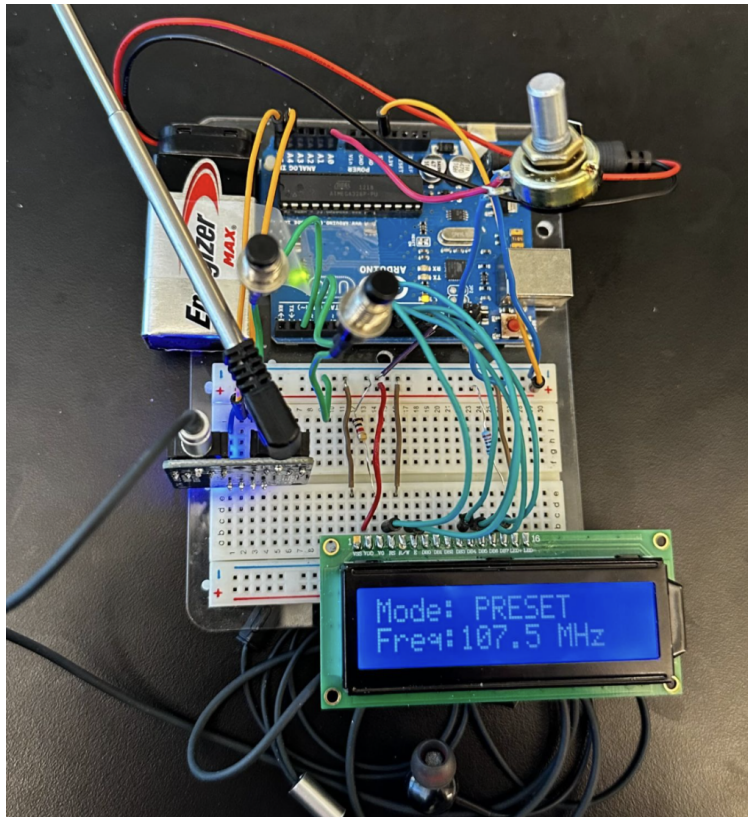


Figure 2: Prototype of the completed Arduino FM radio receiver device

References

- [1] USC Illumin, “Catch a Wave: Radio Waves and How They Work.” <https://illumin.usc.edu/catch-a-wave-radio-waves-and-how-they-work/>, note = Accessed: 2026.
- [2] YouTube, “TEA5767 FM Radio Module for your Arduino (Cheap!).” <https://www.youtube.com/watch?v=yWf9uxL6zgE>, note = Accessed: 2026.
- [3] V. Ceccoli, “Radio Arduino Uno TEA5767.” <https://projecthub.arduino.cc/valerioceccoli/radio-arduino-uno-tea5767-0a4432>. Arduino Project Hub, Accessed: 2026.

Appendix: Arduino Source Code

The following firmware implementation runs on the Arduino UNO microcontroller to manage frequency scaling, user inputs, and LCD interfacing. The codebase was adopted from Ceccoli [3], though a headphone was utilized in place of a speaker for the audio output.

```
1 #include <Wire.h>
2 #include <TEA5767.h>
3 #include <LiquidCrystal.h>
4
5 TEA5767 radio;
6
7 // LCD pins: RS, E, D4, D5, D6, D7
8 LiquidCrystal lcd(7, 8, 9, 10, 11, 12);
9
10 // PINS
11 const int potPin = A0;
12 const int modeButtonPin = 2; // toggle pot / preset mode
13 const int stepButtonPin = 3; // step through presets
14
15 // STATE
16 bool potMode = true; // true = potentiometer mode
17 bool lastModeButtonState = HIGH;
18 bool lastStepButtonState = HIGH;
19
20 // FREQUENCY DATA
21 float currentFreq = 99.9;
22
23 // Preset stations (MHz)
24 float presets[] = { 88.1, 90.1, 92.5, 94.5, 96.5, 99.1, 102.3, 107.5 };
25 const int presetCount = sizeof(presets) / sizeof(presets[0]);
26 int presetIndex = 0;
27
28 // LCD UPDATE
29 void updateLCD(float freq) {
30     lcd.setCursor(0, 0);
31     if (potMode) {
32         lcd.print("Mode: POT ");
33     } else {
34         lcd.print("Mode: PRESET ");
35     }
36
37     lcd.setCursor(0, 1);
38     lcd.print("Freq:");
39     lcd.print(freq, 1);
40     lcd.print(" MHz ");
41 }
42
43 void setup() {
44     Wire.begin();
45     Serial.begin(9600);
46
47     pinMode(modeButtonPin, INPUT_PULLUP);
48     pinMode(stepButtonPin, INPUT_PULLUP);
49
50     lcd.begin(16, 2);
51     lcd.clear();
52
53     radio.setFrequency(currentFreq);
```

```

54   updateLCD(currentFreq);
55
56   Serial.println("FM Receiver Started");
57 }
58
59 void loop() {
60   // MODE TOGGLE BUTTON
61   bool modeButtonState = digitalRead(modeButtonPin);
62
63   if (lastModeButtonState == HIGH && modeButtonState == LOW) {
64     potMode = !potMode;
65     updateLCD(currentFreq);
66   }
67   lastModeButtonState = modeButtonState;
68
69   // POTENTIOMETER MODE
70   if (potMode) {
71     int potValue = analogRead(potPin);
72
73     // Using potValue to calculate new frequency
74     float newFreq = 87.5 + (potValue / 1023.0) * (108.0 - 87.5);
75     newFreq = round(newFreq * 10.0) / 10.0;
76
77     if (abs(newFreq - currentFreq) >= 0.1) {
78       currentFreq = newFreq;
79       radio.setFrequency(currentFreq);
80       updateLCD(currentFreq);
81
82       Serial.print("POT tune: ");
83       Serial.println(currentFreq);
84     }
85   }
86
87   // PRESET MODE
88   else {
89     bool stepButtonState = digitalRead(stepButtonPin);
90
91     if (lastStepButtonState == HIGH && stepButtonState == LOW) {
92       presetIndex++;
93
94       if (presetIndex >= presetCount) {
95         presetIndex = 0; // wrap around
96       }
97
98       currentFreq = presets[presetIndex];
99       radio.setFrequency(currentFreq);
100      updateLCD(currentFreq);
101
102      Serial.print("PRESET tune: ");
103      Serial.println(currentFreq);
104    }
105    lastStepButtonState = stepButtonState;
106  }
107 }

```
